AI-Powered PHP Development in Practice

Building an AI-Augmented Development Workflow That Ships Better Code Faster

Gunnard Engebreth

Longhorn PHP 2026 · Thursday 15 October · Main Ballroom

BEFORE WE START

Get to Green

REPO · ONE AI TOOL

```
Repo    github.com/gunnard/workshop-starter

git pull                # cloned before Tuesday? do this first
composer install
php artisan test        # 15 passed, 3 todos

Agent   opencode (free tier is enough) · claude · Copilot agent mode
```

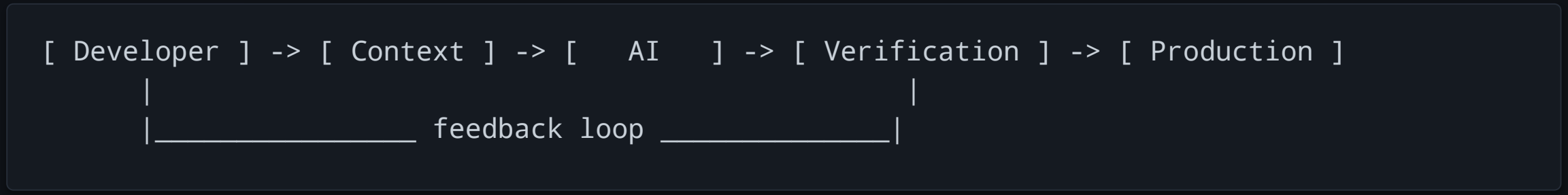
THIS MORNING

Agenda — 9:00 to Noon

TIME	CHAPTER	HANDS-ON
9:00	Settle in, get to green	—
9:05	1 · Why AI changes the job	—
9:13	2 · Environment setup	10 min
9:28	3 · Context engineering	14 min
9:55	**Break**	**10 min**
10:05	4 · Prompt shapes	6 min
10:17	5 · Building a feature, in increments	14 min
10:40	6 · Reviewing AI code	8 min
10:55	**Break**	**10 min**
11:05	7 · Refactoring legacy code	10 min
11:21	8 · Tests that prove something	6 min
11:32	9 · CI guardrails	8 min
11:44	10 · The whole loop, once	7 min
11:52	11 · Close	—

CHAPTER 1

Welcome — Why AI Changes Software Engineering



GOOD AT	BAD AT
boilerplate & scaffolding	your undocumented business rules
pattern-matching common code	knowing what it doesn't know
explaining unfamiliar code	admitting uncertainty
first-draft implementations	novel architectural judgment calls

Environment Setup

SETUP CHECKLIST

```
[ ] PHP 8.3 or newer        php -v
[ ] Composer installed      composer --version
[ ] Repo up to date         git pull
[ ] Packages installed      composer install
[ ] Tests pass              php artisan test
[ ] AI tool started         in the repo folder
```

Zero-Context Baseline

HANDS-ON · 10 MIN

1. Clone the repo, install it, and run the tests. You should see 15 passed and 3 todos.
2. Start your AI tool from inside the workshop-starter folder.
3. Type the prompt below into your agent. Save its answer outside the repo; we compare it in Chapter 3.

PROMPT

```
Explain what this application does, in three sentences,
based only on reading the code.
```

TIME

10 min

DONE WHEN

Your tests pass and you have saved the answer.

STRETCH

Ask which testing framework it uses, and whether its tests have any anti-patterns.

STUCK?

Pair up with a neighbour whose tests pass.

Context Engineering

CONTEXT VALUE HIERARCHY (low -> high leverage)

```
low     Formatting / style rules      <- a linter does this better
        |
        Generic "best practices"      <- the model already knows this
        |
        Architecture decisions + WHY  <- not visible from reading code alone
        |
high    Business rules & vocabulary   <- lives in people's heads otherwise
        Forbidden / footgun patterns  <- costly mistakes, written down once
```

```
# AGENTS.md
## Domain vocabulary
- Tenant: a paying organisation. Seat: a billed user within it.
- seats_limit is the most users a tenant may have; a tenant at its limit cannot add one more.
## Architecture conventions
- Tenancy is by tenant_id on users and issues.
- Seat checks go through Tenant::canAddUser(). Never reimplement the limit inline.
## Forbidden / footguns
- Never return or change an issue without checking it belongs to the requesting user's tenant.
```

Write the Context File

HANDS-ON · 14 MIN

1. Before you write any context file, run the prompt below and save the plan.
2. Read app/Models and app/Http/Controllers, and skim the test names.
3. Create AGENTS.md (or CLAUDE.md for Claude Code) in the repo root. Type in: "Seat checks go through Tenant::canAddUser()." Then add one rule you found.
4. Restart your agent, rerun the prompt, and compare. Then commit the file.

PROMPT

```
Add the ability to invite a user to a tenant.
Propose a plan: which files you'd change and how.
Don't edit any files.
```

TIME

14 min

DONE WHEN

Your file is committed, and you can name one specific way the two plans differ.

STRETCH

Add a second AGENTS.md inside tests/ that applies only to test files.

STUCK?

Run git switch ch4-start to see a finished example.

BREAK

Ten Minutes

BACK AT 10:05

10:05

```
Commit your context file first.
git fetch origin        # while the Wi-Fi is quiet
```

Prompt Engineering for PHP Workflows

PROMPT SHAPES — THREE WE'LL USE TODAY, ALL SIX ON THE CARD

Planning · today	"Propose N approaches with tradeoffs before writing code"
Exploration · today	"Explain how X works, cite files, don't propose changes"
Architecture	"Given constraint C, what pattern fits, and why not the obvious one"
Refactoring	"Preserve behavior. Small steps. What would you change first and why"
Review · today	"Review as a [lens]. Flag [specific risk category]"
Testing	"Given this behavior spec, write tests for the edge cases first"

Reshape a Vague Prompt

HANDS-ON · 6 MIN

1. Pick a real task from your own backlog, or use this one: add CSV export for issues.
2. Ask for it the vague way first: "Add CSV export for issues. Don't edit any files."
3. Then run the prompt below, and compare the two answers.

PROMPT

```
I need to add CSV export for issues. Propose two or three
approaches, with tradeoffs, before writing any code.
Don't edit any files.
```

TIME

6 min

DONE WHEN

You can name one decision the vague prompt made for you without asking.

STRETCH

Add a lens: "as an engineer worried about speed — this table could have millions of rows."

Building Features — The Iterative Loop

THE LOOP, APPLIED

```
A vague ticket
   |
   v
Ask the agent for clarifying questions first --> the gaps get filled
   |
   v
Ask for 2-3 approaches, then pick one
   |
   v
Increment 1: the route, its input, who may use it  --> review --> commit
Increment 2: closing the issues                    --> review --> commit
Increment 3: the response, and its tests           --> review --> commit

Stuck? Ask: which layer is wrong? Am I fixing the cause, or just the symptom?
```

Bulk-Close, in Increments

HANDS-ON · 14 MIN

1. Type the ticket below into your agent, and ask it for clarifying questions first. Answer them.
2. Ask for two or three approaches, and pick one.
3. Ask for increment 1 only: the route, its input, and who may use it. Review the diff, run the tests, commit.
4. If there is time, do increments 2 and 3 the same way, one at a time.

TICKET

```
Users want to be able to close multiple issues at once
instead of one at a time. Add it to the issues API.
```

TIME

14 min

DONE WHEN

Increment 1 is committed and all the tests pass.

STRETCH

Halfway through, change the brief: users can undo within 10 seconds.

STUCK?

Routes go in routes/web.php (there is no api.php). At minute 12, commit and stop: Chapter 6 reviews a finished version.

Reading AI Code — Systematic Review

REVIEW CHECKLIST — ORDER MATTERS

1. Architecture fit	-- is this in the right layer?
2. Correctness	-- does it do what was actually asked, incl. edge cases?
3. Security / authz	-- dedicated pass, don't fold into "correctness"
4. Naming / complexity	-- only after 1-3 are clean
5. Performance	-- N+1s, missing eager loads, unneeded DB::raw
6. Maintainability	-- would a new hire understand this in six months?

Review a Finished Bulk-Close

HANDS-ON · 8 MIN

1. Read the diff below. You don't need to switch branches.
2. Go through the review checklist in order, and write one line for each point.
3. Then answer: what happens if the list includes an issue from another tenant?
4. Compare it with what you built in Chapter 5.

THE DIFF

```
git fetch origin
git diff origin/ch4-start origin/ch6-start
```

TIME

8 min

DONE WHEN

You have written six lines and answered the question in step 3.

STRETCH

Fix show() and switch its test on. Then fix the N+1 query in index().

STUCK?

Start at point 3 of the checklist. Still stuck? Ask me for the hint.

BREAK

Ten Minutes

BACK AT 11:05

11:05

```
Commit whatever you have.
The legacy file is the same on every branch — nobody is behind.
```

AI-Assisted Refactoring of Legacy Code

LEGACY REFACTOR SEQUENCE

```
Do tests exist, and do they cover the code you're about to change?
   | no
   v
First write characterization tests: record what it does now, bugs included
   |
   v
Then pull out ONE thing: a method, a service, or a class
   |
   v
Run the tests. Still passing? Commit.
   |
Repeat. Fix bugs in a separate commit whose message says it changes behaviour.
```

Characterize, Then Extract One Thing

HANDS-ON · 10 MIN

1. Ask your agent which behaviours the existing tests check. Note the gaps.
2. Ask for tests that record what the code does now, bugs included. Run them, then commit.
3. Use the prompt below to move one piece of logic out. Run the tests, commit.

PROMPT

```
Extract the counting logic into a new ReportSummary class.
Preserve behavior, including existing bugs, unless I tell
you otherwise. One small, reviewable change.
```

TIME

10 min

DONE WHEN

You have two commits, and every test passes.

STRETCH

Fix one bug in its own commit, with a message saying it changes behaviour.

STUCK?

The legacy controller is the same on every branch, so nobody is behind.

AI-Generated Testing

TEST QUALITY CHECKLIST

- [ ] Do the tests cover edge cases and failures, not just the happy path?
- [ ] Do they check what the code does, rather than how it does it?
- [ ] Would this suite have caught the Chapter 6 bulk-close flaw?
- [ ] Break one boundary on purpose (for example < to <=). Does a test fail?
- [ ] Make a method always return the same value. Does a test fail?

Does Your Test Guard Anything?

HANDS-ON · 6 MIN

1. Delete the ->todo() on the seat-limit test. Run the tests and watch it fail.
2. Fix Tenant::canAddUser() so the tests pass. Commit the fix and the test.
3. Break it on purpose: change < back to <=. A test should fail. Then undo it.
4. Make the method just return false;. Does a test fail? If not, use the prompt below.

PROMPT

```
Write Pest tests for Tenant::canAddUser(). Edge cases
and boundaries first: below, at and over the limit,
a limit of zero. Then the happy path.
```

TIME

6 min

DONE WHEN

Your return false; version makes at least one test fail.

STRETCH

Write a test that would catch ch6-start closing another tenant's issues.

STUCK?

Run git switch ch8-start. The bug is fixed there, so start at step 3.

CI/CD Guardrails — Catching Hallucinations Before They Ship

CI STAGE	ANSWERS WHICH EARLIER FAILURE?
Static analysis	Catches invented methods and wrong types (Ch 4, 6), but it trusts every @var you write.
Style / Pint	Keeps formatting out of review, so reviewers can focus on the Ch 6 checklist.
Test suite gate	Re-checks the Ch 6 and Ch 8 bugs on every push.
Security audit	Finds vulnerabilities published after you locked your dependencies.
Mutation (gated)	Tests your tests: Ch 8's "green suite that doesn't mean much". Not in this repo yet.

Make CI Catch Something

HANDS-ON · 8 MIN

1. Run composer ci. Everything should pass.
2. Make a branch, then put one bug back with its test on: < back to <= in canAddUser(), or remove show()'s tenant check.
3. Run composer ci again. Which stage failed? Which stages could never have caught it?
4. Undo your change, and check that everything passes again.

WHAT CI RUNS, LOCALLY

```
composer ci        # PHPStan, Pint, Pest
```

TIME

8 min

DONE WHEN

The tests failed, then passed again, and you can say why PHPStan couldn't catch it.

STRETCH

On ch6-start, delete the @var line in IssueController.php. PHPStan level 9 then finds 5 errors.

STUCK?

Run git switch solution. Every bug is fixed there and every test is on.

Loop Drill — The Whole Loop, Once

THE FULL LOOP, ONE MORE TIME

```
Check context -> ask questions -> plan -> build in small steps ->
review with the checklist -> edge-case tests first -> CI -> merge
```

One Ticket, the Whole Loop

HANDS-ON · 7 MIN

1. Check your context file: does this ticket need anything added?
2. Have your agent ask clarifying questions, then propose a few approaches. Pick one.
3. Build one increment, and review it with the Chapter 6 checklist.
4. Write one edge-case test before the code it checks.

TICKET

```
Tenant admins want to see how many of their issues
are open, in progress, and closed.
```

TIME

7 min

DONE WHEN

You have a plan you'd approve, one reviewed increment, and one edge-case test.

STRETCH

The test that fails if in_progress gets counted as closed — the way the legacy report does.

STUCK?

Go back: context (3), increments (5), checklist (6), edge cases (8).

What’s Changing, and What Isn’t

WHAT'S CHANGING	WHAT ISN'T
Which tools we use	“whoever merges owns the bug”
How context gets loaded	Checking the work before it ships (Ch 6–9)
How long agents work without checking in	Context beats clever prompting (Ch 3)

BEFORE LUNCH

Take It Home

TAKE WITH YOU	DO ON MONDAY
Context beats cleverness	Write a one-page context file for your repo
Plan before code	Put three plan-first prompts in your team's library
Review in order; security gets its own pass	Print the review checklist
Characterize before you refactor	Characterization tests before your next refactor
Checks belong in CI	Make CI a required check

`github.com/gunnard/workshop-starter`

Prompt library and checklists: docs/prompt-library.md in the same repo

Next in this room, 1:00 — **Eric Mann · OWASP Top 10**